

Lab Report - EE-170L Computer Programming (Lab 09)

NAME: _MAAZ WAHAB _____

REG NO: _BF25NWELE0719 _____

SEMESTER: 2nd

LAB: 9 (Programming)

DATE: __6 may 2026_____

1. Objective

The main goal of this lab is to practice using **functions in C++**. Functions are one of the most important concepts in programming because they allow us to break down tasks into smaller reusable blocks. In this lab, we focus on:

- **Introduction to Functions**
- **Function Prototype**
- **Function Declaration and Call**
- **Functions with Parameters and Return Values**

By the end of this lab, students should be able to:

- Write programs that declare and call functions.

- Understand the difference between void functions and functions with return values.
- Apply functions to solve problems like summation and averaging.
- Recognize how parameters make functions flexible and reusable.

2. Theory

2.1 Introduction to Functions

Functions let us divide a program into named sections that can be reused. A function has:

- A **name identifier**
- **Parameters** (optional inputs)
- A **return type** (or void if nothing is returned)

The most common function is `main()`, which you have used throughout labs. Other functions can be defined by the programmer to perform specific tasks.

2.2 Function Prototype

A **function prototype** describes the function's name, parameters, and return type. It tells the compiler what kind of calls are valid.

Syntax:

```
cpp
```

```
returnType functionName(parameter_list);
```

Examples:

```
cpp
```

```
int main();
```

```
float divide(int a, int b);
```

```
void square(float x);
```

2.3 Function Declaration and Call

- **Declaration:** Introduces the function to the compiler.
- **Definition:** Contains the actual code.

- **Call:** Executes the function from main() or another function.

Functions can take **no parameters**, **single parameters**, or **multiple parameters**.

3. Programs and Outputs

Task 1: Function Introduction and Declaration

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
// Function prototype
```

```
void greet();
```

```
int main() {
```

```
    // Function call
```

```
    greet();
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
void greet() {
```

```
    cout << "Welcome to EE-170L Lab on Functions!" << endl;
```

```
}
```

Output:

Code

Welcome to EE-170L Lab on Functions!

Task 2: Basic Function Declaration and Call

```
cpp

#include <iostream>

using namespace std;


// Function prototype

void sayHello();


int main() {

    // Function call

    sayHello();

    return 0;

}


// Function definition

void sayHello() {

    cout << "Hello, World!" << endl;

}
```

Output:

Code

Hello, World!

Task 3: Function with Parameters and Return Value

```
cpp

#include <iostream>

using namespace std;
```

```
// Function prototype
int calculateSum(int arr[], int size);

int main() {
    int numbers[10]; // Array to store 10 integers

    // Taking input from user
    cout << "Enter 10 integers:" << endl;
    for(int i=0; i<10; i++) {
        cin >> numbers[i];
    }

    // Function call to calculate sum
    int sum = calculateSum(numbers, 10);

    // Calculate average
    float average = sum / 10.0;

    // Display results
    cout << "Sum of elements = " << sum << endl;
    cout << "Average of elements = " << average << endl;

    return 0;
}
```

```
// Function definition

int calculateSum(int arr[], int size) {

    int sum = 0;

    for(int i=0; i<size; i++) {

        sum += arr[i]; // Add each element to sum

    }

    return sum; // Return total sum

}
```

Sample Output:

Code

Enter 10 integers:

1 2 3 4 5 6 7 8 9 10

Sum of elements = 55

Average of elements = 5.5

4. Discussion

This lab showed how functions make programming easier and more efficient. Without functions, we would have to repeat code blocks for similar tasks. With functions, the same task can be done in a reusable way.

- **Void functions** are best when we just want to display or perform an action.
- **Functions with return values** are useful when we need results for further calculations.
- **Parameters** make functions flexible, allowing them to work with different inputs.

By practicing functions, I learned how to modularize code and improve readability.

5. Conclusion

In this lab, I wrote programs using functions with and without parameters. I learned how each function works and where it can be used.

- Functions improve modularity and reusability.

- Function prototypes help the compiler understand valid calls.
- Parameters and return values make functions powerful for solving problems.

This lab improved my understanding of functions and helped me practice writing programs that are structured and efficient.